

Alpha-GPT: Internal Research Note

July 1, 2026

Abstract

Finding a portfolio of alpha expressions that ranks future cross-sectional returns is a search over an intractable, doubly-exponential space of programs against a black-box objective known only by backtesting. We replace brute-force enumeration with *economic-reasoning-guided search*: a multi-agent LLM debate proposes economically motivated expressions (large, informed jumps through the space), genetic programming refines them locally, and the debate’s cross-review plus a structural verifier prune weak candidates cheaply, so the expensive objective is queried only on promising ones. The method specializes the *system*, not the model – a general LLM already holds the asset-pricing reasoning we need, so we leave its weights untouched and wrap it in a task-specific harness whose only levers are a decomposition (reason about the economic mechanism, then commit to a formula) and test-time compute (the debate). We deliberately target one modular stage of the alpha pipeline – the generator – holding data, portfolio construction, and backtesting fixed: this attributes any measured edge to the alphas themselves and makes the generator a drop-in module for existing quant infrastructure. Whether this latent economic skill beats data-mining is the empirical question our random, single-shot, and genetic-programming baselines are built to test.

1 Foundations: the objects we manipulate

1.1 Data: panels, universe, splits

The market is represented as a library of *panels*. A panel for data field k is a matrix

$$X^{(k)} \in \mathbb{R}^{T \times N}, \quad X_{t,i}^{(k)} = \text{value of field } k \text{ for stock } i \text{ on day } t,$$

indexed by trading date t (rows) and security identifier (PERMNO) i (columns). All fields share one date-by-stock grid, so they can be combined cell-wise. Missing cells are **NaN** and every operator is NaN-safe.

The investable set on day t is the point-in-time universe $\mathcal{U}_t \subseteq \{1, \dots, N\}$ (roughly 1,500 names per year, re-selected annually by trailing market capitalization; survivorship-free, so names enter and exit at delisting). A cell is valid only if $i \in \mathcal{U}_t$ and the stock actually traded that day.

The prediction target. For each stock and day we define the *forward return*

$$y_{t,i} = r_{t+1,i} = \frac{P_{t+1,i}}{P_{t,i}} - 1,$$

the next-day total return (delisting returns embedded). Forward returns are masked to the universe and the last in-sample observation is blanked so no label leaks across a split boundary.

Splits. The calendar is partitioned once:

$$\underbrace{2010-2017}_{\text{train}} \mid \underbrace{2018-2020}_{\text{validation}} \mid \underbrace{2021-2025}_{\text{test}}.$$

Selection of everything (which expressions, which combiner, which weights, which hyperparameters) happens on train/validation. The test split is touched exactly once, by the final portfolio backtest. This constraint reappears in Section 2 as an explicit feasibility condition on the optimizer.

Data sources and what we can add. Table 1 lists the live sources and natural extensions. The key axes are *source*, *type*, *native frequency*, and the orthogonal information each adds.

Source	Type	Frequency	Notes / fields
CRSP (CIZ)	price / volume	daily	OHLC, returns (delisting-adjusted), volume, shares, market cap, bid/ask. The price-volume base.
Compustat WRDS ratios	+ fundamentals	quarterly	~17 characteristics (profitability, leverage, B/M, E/P, accruals, growth), point-in-time via earnings-announcement date <code>rdq</code> , forward-filled to daily.
<i>Natural extensions (orthogonal information families):</i>			
OSAP (Chen– Zimmermann)	published anomaly signals	monthly	~200 firm-level signals; ingestion is implemented but not currently loaded into the live universe.

Table 1: Data sources, types, native frequencies, and extensions. Lower-frequency sources are forward-filled onto the daily grid with a point-in-time release lag, so mixing frequencies never introduces look-ahead.

1.2 An alpha expression

Definition 1.1 (Alpha expression). An *alpha expression* e is a well-typed symbolic tree whose leaves are *terminals* (data panels, drawn from a finite menu $\mathcal{T}$) and whose internal nodes are *operators* (drawn from a finite set $\mathcal{O}$). Evaluating the tree produces a *signal panel*

$$\alpha = e(X) \in \mathbb{R}^{T \times N}, \quad \alpha_t = (\alpha_{t,i})_{i \in \mathcal{U}_t} \in \mathbb{R}^{N_t}$$

where α_t is the day- t cross-section of scores and $N_t = |\mathcal{U}_t|$.

The operator set is small and fixed (the “alpha language”):

- **Time-series** (per stock, along time; curried with fixed windows $w \in \{5, 10, 20, 60\}$): `ts_mean`, `ts_std`, `ts_delta`, `ts_rank`, `ts_min`, `ts_max`, `ts_returns`, and the binary `ts_corr`.
- **Cross-sectional** (per day, across stocks): `cs_rank`, `cs_zscore`.
- **Element-wise**: `add`, `sub`, `mul`, `safe_div`, `log_abs`, `abs_val`, `sign`, `neg`.

A concrete example, “recent return scaled by volatility, ranked cross-sectionally,” is

$$e = \text{cs_rank}(\text{safe_div}(\text{ts_returns_20}(\text{close}), \text{ts_std_20}(\text{returns}))).$$

Why the space is intractable. Let b be the typical operator arity and D the maximum tree depth. The number of admissible expressions grows roughly as

$$|\mathcal{E}| \sim (|\mathcal{T}| + |\mathcal{O}|)^{\Theta(b^D)}, \quad (1)$$

doubly exponential in depth. The search is over *programs* (terminals, operators, and nesting), not over feature pairs, so direct enumeration is infeasible and a guided search is required.

Remark 1.1 (Formula vs. code representation). The tree above fixes one *level* of representation – a composition of the bounded operator set $\mathcal{O}$, i.e. a symbolic formula. A deeper alternative, central to concurrent work (CogAlpha; Liu et al., 2026), lets the generator emit general *code* (intermediate variables, regime logic, transforms outside $\mathcal{O}$), reaching the strictly larger $\mathcal{E}_{\text{code}} \supset \mathcal{E}_{\text{formula}}$ and matching what LLMs do well. We default to the DSL because it is fast, parseable, and open to the free genetic search of Section 4, whereas code needs sandboxing, timeouts, and a look-ahead test, evaluates more slowly, and forfeits free GP – pushing toward far smaller candidate counts. Since only this backend changes, “generate code” also drops in as a third arm for an all-else-equal comparison the literature lacks.

1.3 From a signal to cross-sectional bets

A signal is used *cross-sectionally*: each day we rank the stocks by α_t and bet that high-score names outperform low-score names. Formally, on day t assign each valid stock to one of $Q = 5$ quantiles by the rank of its score,

$$q_{t,i} = \left\lceil Q \cdot \frac{\text{rank}(\alpha_{t,i})}{N_t} \right\rceil \in \{1, \dots, Q\},$$

(ranking with tie-breaking so the bins never collapse). The long–short quintile portfolio is long the top bucket and short the bottom bucket, equal-weighted, and its realized next-day return is

$$R_t^p = \underbrace{\frac{1}{|L_t|} \sum_{i \in L_t} y_{t,i}}_{\text{long top quintile}} - \underbrace{\frac{1}{|S_t|} \sum_{i \in S_t} y_{t,i}}_{\text{short bottom quintile}}, \quad \begin{aligned} L_t &= \{i : q_{t,i} = Q\}, \\ S_t &= \{i : q_{t,i} = 1\}. \end{aligned} \quad (2)$$

Because positions are determined by *rank*, only the cross-sectional ordering of α_t matters, not its scale; any monotone transform of a signal is the same bet.

1.4 Measuring one signal: the information coefficient

The standard, scale-free measure of how well a signal ranks future returns is the *information coefficient*: the daily Spearman rank correlation between the score and the forward return.

Definition 1.2 (IC, ICIR). For expression e with signal $\alpha = e(X)$,

$$\text{IC}_t(e) = \rho_{\text{Spearman}}(\alpha_t, y_t) = \rho_{\text{Pearson}}(\text{rank}(\alpha_t), \text{rank}(y_t)), \quad \text{IC}(e) = \frac{1}{T} \sum_{t=1}^T \text{IC}_t(e), \quad (3)$$

and the information ratio of the signal is

$$\text{ICIR}(e) = \frac{\overline{\text{IC}_t}}{\text{std}_t(\text{IC}_t)}, \quad t\text{-stat} = \text{ICIR}(e) \sqrt{T}. \quad (4)$$

IC measures average directional accuracy (a daily IC persistently above ≈ 0.02 is economically meaningful); ICIR measures its stability / signal-to-noise. We also track *turnover*, the mean day-over-day change in the cross-sectional percentile rank,

$$\tau(e) = \frac{1}{T} \sum_t \frac{1}{N_t} \sum_i |u_{t,i} - u_{t-1,i}|, \quad u_{t,i} = \frac{\text{rank}(\alpha_{t,i})}{N_t}, \quad (5)$$

a proxy for rebalancing cost.

Remark 1.2 (Why IC and not raw PnL for ranking candidates). IC in (3) is a rank statistic, so it is robust to outliers and immune to the monotone-transform ambiguity noted in Section 1.3. It is also cheap to compute in bulk, which matters because the search evaluates enormous numbers of candidates (Section 4).

1.5 Combining many signals into a portfolio

A single alpha is weak; the edge comes from combining many weak, weakly-correlated signals into a portfolio (this is the *breadth* that Section 2 shows matters most). Given a selected set $S = \{e_1, \dots, e_M\}$, each signal is first *oriented* and *standardized*:

$$s_t^{(j)} = \text{sign}(\text{IC}^{\text{val}}(e_j)) \cdot z(\alpha_t^{(j)}), \quad z(\alpha_t)_i = \frac{\alpha_{t,i} - \text{mean}(\alpha_t)}{\text{std}(\alpha_t)}, \quad (6)$$

so every signal points the “profitable” way (sign chosen on validation, never test) and lives on a comparable scale. The composite is then a weighted blend $g_t = \sum_j w_j s_t^{(j)}$. Because portfolio construction is not the research question (Section 2 expands on this), we use the simplest robust weights: the default is **equal-weight** ($w_j = 1/M$), with **IC-weighting** ($w_j \propto |\text{IC}^{\text{val}}(e_j)|$) as a variant. Both use only validation-derived signs and weights, so no test information enters construction.

1.6 The objective: out-of-sample Sharpe

Running the composite g through the long-short map (2) yields a daily portfolio return series R_t^p . The headline objective is its annualized Sharpe ratio on the test split:

$$\boxed{\text{SR} = \sqrt{252} \frac{\text{mean}_t R_t^p}{\text{std}_t R_t^p}} \quad (7)$$

We also report annualized return, maximum drawdown $\text{MDD} = \min_t (C_t / \max_{s \leq t} C_s - 1)$ with $C_t = \prod_{s \leq t} (1 + R_s^p)$, and the portfolio’s own IC/ICIR/ τ . Equation (7) is the scalar that Part I maximizes.

2 Part I: the optimization problem

The generator must choose a set $S \subseteq \mathcal{E}$ of expressions that, combined into a long-short portfolio, maximizes out-of-sample Sharpe. Everything downstream is fixed by scope: the combiner is not fitted, only picked from a small menu of robust weights on validation, so there is no inner portfolio problem and S is the sole decision variable:

$$\boxed{\begin{array}{ll} \max_{S \subseteq \mathcal{E}} & \mathbb{E}[\text{SR}_{\text{test}}(\text{portfolio}(S))] \quad \text{s.t.} \quad \begin{cases} \text{every } e \in S \text{ is parseable and passes the verifier,} \\ S \text{ and the combiner are chosen on train/validation only.} \end{cases} \end{array}} \quad (8)$$

The expectation matters because the optimizer never sees SR_{test} ; it maximizes a validation surrogate, and test is read exactly once. That validation–test gap is the central statistical risk, reduced by the consistency filter (Section 4) and the debate’s cross-review. The problem is hard for two reasons – $\mathcal{E}$ is doubly exponential (1) and each evaluation is an expensive backtest – so we can neither enumerate nor differentiate, only *search* (Part II).

What the objective rewards: skill and breadth. For a dollar-neutral long–short book the portfolio return *is* the active return, so its Sharpe equals the *information ratio*, and Grinold’s fundamental law factors that same number into the two levers a proposer controls:

$$\boxed{\text{SR} = \text{IR} \approx \underbrace{\text{IC}}_{\text{skill}} \cdot \sqrt{\underbrace{\text{breadth}}_{\# \text{ independent bets}}} \quad (9)}$$

Skill enters linearly but breadth only under a square root, so many diverse, modest-IC alphas beat a few strong ones. The search therefore maximizes the *number of independent* skilled alphas rather than individual quality, and the combiner keeps S large with no top- k cap – provided the added alphas are genuinely independent, not restatements of the same bet. “Skill $\times$ breadth” is also the axis along which the proposal mechanisms of Part II differ (Section 3.2).

Remark 2.1 (Combiner: equal weight, not fitted tangency). The Sharpe-maximizing weights for a fixed S are the tangency portfolio $w^* \propto \Sigma^{-1}\mu$. With many correlated, noisy alphas the sample Σ^{-1} is ill-conditioned and w^* overfits; equal weight is its maximal-shrinkage limit, beats sample tangency out of sample, and keeps any measured edge attributable to the alphas rather than to combiner tuning.

3 Part II: how the search works

Since (8) can be neither enumerated nor differentiated, we search. The search is a generate-and-test loop built to spend cheap effort widely and the expensive true objective narrowly: two complementary moves generate candidates, and a cascade of increasingly expensive checks filters them, so the real backtest runs only on the few that survive.

3.1 Propose globally, refine locally, prune cheaply

1. **Propose (global moves).** The multi-agent debate takes an economically reasoned hypothesis and writes *complete* expressions for it. Because it draws on the model’s pretrained economic knowledge, it lands directly in plausible regions of $\mathcal{E}$ instead of wandering in from random starts – large, informed jumps.
2. **Refine (local moves).** Each proposal seeds genetic programming, which mutates and recombines it (subtree crossover, point mutation) to hill-climb train IC in a small neighborhood. One informed seed thus spawns many nearby variants at no extra model cost – small, cheap steps.
3. **Prune (cheap first, expensive last).** Running the true backtest is the costly step, so cheaper checks come first, in order: the debate’s cross-review drops economically weak ideas, the structural verifier discards anything that fails to parse, evaluate, or vary, and a fast approximate IC ranks what remains. Only the top survivors reach the full backtest.

The division of labor is the point: the LLM makes the large, economically informed jumps a local search never could, and GP does the cheap refinement around them that the LLM would waste calls doing by hand. The proposer is *fixed* – it does not learn from which past alphas backtested well.

3.2 Why multi-agent debate

Diversity, then critique. Search quality is set by the *best* candidate found under a fixed proposal budget. One model sampled many times tends to return the same few ideas reworded, so most of the budget re-draws a single mode; three persona-conditioned proposers (Momentum, MeanReversion, Fundamental) instead pull toward *different* economic mechanisms, so the same budget covers more of $\mathcal{E}$ and lifts the expected best. Debate then adds what a plain sampler cannot: each agent critiques the others (cross-review), killing weak proposals before any of them costs a backtest. Debate thus buys both *exploration* (diverse proposals) and *verification* (critique).

Why debate and not the other inference-time methods.

- **Self-consistency / majority vote** collapses many reasoning chains onto one consensus answer; we want the opposite – a *diverse set* of distinct alphas.
- **Best-of- N** samples one model and keeps the top-scored; with no persona diversity and no critique, its proposals cluster and it explores narrowly.
- **MCTS / tree-of-thought** needs a cheap, accurate value estimate for partial programs to steer the tree; here the only ground truth is the expensive backtest, so there is nothing cheap to steer by.

The hypothesis: debate is the only mechanism with both skill and breadth. The fundamental law (9) says a good proposer needs *skill* (high per-alpha IC) and *breadth* (many independent alphas), and the skill that counts is *out-of-sample*. This is exactly where an economic prior earns its keep: brute-force enumeration and genetic programming can push up *in-sample* IC, but with no economic reasoning to anchor them their best fits are usually overfit and fade out of sample. Table 2 lays out where we *expect* each mechanism to land: the priorless methods fall short on genuine skill, breadth, or both, whereas debate is built for both – genuine skill from economic priors plus cross-review, breadth from cheap, diverse proposals. These are predictions; the ablation grid of Section 3.3 measures whether they hold.

3.3 Baselines and the ablation grid

To test these predictions we run the debate against baselines that strip out the economic prior and the multi-agent structure, each through an *identical* downstream pipeline (verify, evaluate, filter, combine, backtest; Part III), so any difference traces to the proposal mechanism alone. Crossing the mechanism with a GP on/off toggle gives the grid in Table 3: reading down a column isolates the economic prior (random vs. single-shot LLM vs. debate), reading across a row isolates local GP amplification (off vs. on).

The down-column contrast – does an economic prior, and *multi-agent* structure, beat the mechanical baselines? – is the project’s central hypothesis; the sharpest single test is **single-agent-gp** vs. **debate-only**, which at matched downstream effort separates structured multi-agent reasoning from merely spending more compute on one proposer.

Mechanism	Skill	Breadth	Why we expect this
Random trees	none	high	unlimited cheap trees, but no economic prior – the signals are meaningless
Brute-force enum.	low	very low	no economic prior, so its high in-sample IC is mostly overfit; also reaches only a shallow sliver of $\mathcal{E}$
Genetic programming	low	low	hill-climbs in-sample IC with no economic grounding, so its winners tend to overfit; explores only locally
Single-shot LLM	moderate	moderate	an economic prior gives a genuine but unrefined edge; one call clusters on a theme and gets no critique
Multi-agent debate	high	high	economic priors plus cross-review favor genuinely predictive alphas; diverse personas widen coverage

Table 2: *Hypothesized* skill and breadth by proposal mechanism – predictions, not measurements. *Skill* is genuine *out-of-sample* per-alpha IC: the priorless mechanical methods can fit the training data, but with no economic reasoning to lean on that fit tends to overfit rather than generalize. *Breadth* is the number of independent alphas the mechanism can supply. Only debate is designed for both. Section 3.3 tests these predictions.

Proposal mechanism	GP off	GP on	LLM cost / round
Random trees (noise floor)	random	random-gp	free
Single-shot LLM (one call)	single-agent	single-agent-gp	~1 call
Multi-agent debate (the method)	debate-only	full	~10–12 calls

Table 3: Ablation grid: proposal mechanism $\times$ GP toggle (cell labels are the implemented run modes). The downstream verify / evaluate / filter / combine / backtest path is identical across all six cells, so differences come only from how candidates are proposed and amplified.

Remark 3.1 (Debate + GP). Adding GP on top of the debate pushes breadth further – each economically-reasoned seed spawns many verified neighbors – but GP’s crossover and mutation are economically blind, so the variants can dilute the skill that made the seeds good. Debate + GP thus trades a little per-alpha skill for breadth; whether that is net-positive is what the GP-on column of Table 3 answers.

4 Part III: numerics and algorithms

The concepts of Part II become three algorithms operating on two objects – the *context* $\mathcal{H}$ (the hypothesis under exploration plus the resolved terminal menu $\mathcal{T}$) and the *book* $\mathcal{P}$ (every expression found so far with its cached train/validation metrics) – and four components: three **debate agents** (Momentum, MeanReversion, Fundamental), a **moderator** that synthesizes their proposals, a **GP operator** that mutates and recombines seeds, and a **verifier** that structurally filters candidates (parse, depth, eval, coverage, non-constant) to keep the book clean at scale.

4.1 The multi-agent debate (the proposer)

Each of the two stages is a draft / cross-review / revise loop over the three agents, closed by a moderator. Stage 1 produces hypotheses (no formulas yet); Stage 2 turns them into parseable expressions.

Algorithm 1 Two-stage debate: idea $\rightarrow$ formula (one full proposal round)

Require: trading idea / context $\mathcal{H}$, terminal menu $\mathcal{T}$, agents $\{1, 2, 3\}$, moderator Φ

Stage 1: idea debate

- 1: each agent i **drafts** a structured hypothesis (mechanism, direction, proxies, filters, normalization)
- 2: each agent **cross-reviews** the others on 8 dimensions (mechanism, clarity, payoff, direction, subfactor, filter, normalization, implementability), decision $\in \{\text{accept}, \text{accept-with-revision}, \text{reject}\}$
- 3: each agent **revises** its hypothesis given the reviews
- 4: $\mathcal{H}^* \leftarrow \Phi(\text{revised hypotheses})$ $\triangleright$ moderator synthesizes ≈ 3 hypotheses

Stage 2: formula debate

- 5: each agent **drafts** 1–2 expressions per hypothesis over $\mathcal{T}$
 - 6: each agent **cross-reviews** on $\{\text{faithfulness}, \text{implementability}, \text{robustness}, \text{novelty}, \text{simplicity}\}$
 - 7: each agent **revises**; **parse-gate**: drop any expression that does not compile against $(\mathcal{T}, \mathcal{O})$
 - 8: **return** seed pack $\leftarrow \Phi(\text{parseable expressions})$ $\triangleright$ moderator selects ≈ 8 diverse seeds
-

Cost is roughly 10–12 model calls per round (3 agents $\times$ 3 phases $\times$ 2 stages plus 2 moderator calls). This is the expensive, high-quality proposal path; the factory can also use a single autonomous generator call when scale matters more than per-idea quality.

4.2 The outer search loop (factory + GP amplification)

Algorithm 2 Outer search: propose $\rightarrow$ amplify $\rightarrow$ verify $\rightarrow$ evaluate $\rightarrow$ store

Require: number of rounds N , train/val splits, primitive set pset

- 1: $\mathcal{P} \leftarrow \emptyset$ $\triangleright$ the book / database, deduplicated on expression hash
 - 2: **for** $n = 1, \dots, N$ **do**
 - 3: seeds $\leftarrow \text{PROPOSE}(\mathcal{H}_n)$ $\triangleright$ Algorithm 1, or single-agent / random
 - 4: candidates $\leftarrow \text{seeds} \cup \text{GP-AMPLIFY}(\text{seeds})$ $\triangleright$ local search around each seed
 - 5: **for** each e in candidates **do**
 - 6: **if not** $\text{VERIFY}(e)$ **then continue** $\triangleright$ parse, depth, eval, coverage, non-constant
 - 7: **end if**
 - 8: compute $\text{IC}^{\text{train}}(e), \text{IC}^{\text{val}}(e), \text{ICIR}, \text{SR}^{\text{val}}, \tau$ $\triangleright$ fast approximate metrics
 - 9: insert e into $\mathcal{P}$
 - 10: **end for**
 - 11: **end for**
 - 12: **return** $\mathcal{P}$
-

GP-AMPLIFY runs DEAP genetic programming seeded with the parsed proposals (subtree crossover, point mutation, depth limit 6; fitness = mean train IC on a date-subsample for speed), so each informed seed spawns many verified neighbors with no extra model calls.

Algorithm 3 From the book to one out-of-sample number

Require: book $\mathcal{P}$, splits (val, test), combiners $\mathcal{C}$

- 1: $S \leftarrow \{e \in \mathcal{P} : \text{IC}^{\text{train}}(e) \cdot \text{IC}^{\text{val}}(e) > 0 \text{ and } |\text{IC}^{\text{val}}(e)| \geq \epsilon\}$ ▷ consistency filter, no top- k cap
 - 2: orient + z-score each $e \in S$ on test via (6) ▷ signs from validation
 - 3: **for** each combiner $c \in \mathcal{C}$ **do**
 - 4: $g \leftarrow \text{combine}(S, c)$; $R^p \leftarrow$ long-short quintile of g on test ▷ (2)
 - 5: record SR, IC, ICIR, τ , MDD ▷ (7)
 - 6: **end for**
 - 7: choose the headline combiner by **validation** Sharpe ▷ not by max test Sharpe
 - 8: **return** the test metrics of the chosen combiner
-

4.3 Selection and the out-of-sample backtest

Consistency filter, not top- k . Selecting the highest-validation-IC alphas tends to pick the most overfit ones; instead we require the sign to agree across train and validation ($\text{IC}^{\text{train}} \cdot \text{IC}^{\text{val}} > 0$), which drops the lucky ones while keeping the set large. Even the combiner is chosen on validation, so no decision peeks at test.

4.4 Evaluation speed and cost

IC is always the exact per-day Spearman (3) – one metric used both to rank candidates during the search and to report the final portfolio. Sharpe comes in two forms: a fast rank-weighted proxy for ranking candidates, and the exact quintile long-short backtest (2) for the headline number, run only on the filtered survivors. Everything parallelizes; at roughly \$0.00008 per alpha, 10^3 alphas cost about \$0.06 in ≈ 13 minutes and scale to 10^6 , so wall-clock, not money, binds – which is why cheap GP amplification per expensive proposal matters.